Rough initial exploration

Improving multi-agent systems with latent communication

Original reference papers from Yusen:

- https://arxiv.org/pdf/2310.06272
- https://arxiv.org/pdf/2505.15778
- https://arxiv.org/pdf/2501.14082

LatentMAS & Intelat already implements this for multi-agent systems:

- https://github.com/Gen-Verse/LatentMAS (LatentMAS; uses KV cache communication for inter agent, and last state for inside agent)
- https://arxiv.org/abs/2511.09149 (Interlat; uses last state for everything)
    - Compresses steps a lot, too

Papers that use KV-cache as communication medium:
https://arxiv.org/abs/2510.03215 (model fusion! Not A to B communication)
https://arxiv.org/abs/2510.03346 (KVcomm; A to B communication)
https://arxiv.org/abs/2411.02820 (droidspeak, more OG)

Soft-token vs. continuous thought

- Continuous thought often requires extra steps (residuals are not enough to be in distribution)
- Soft-token is easier to just plug back into the transformer
    - But diverges after too many steps (because OOD)

Ideas:

- A huge survey on multi-agent systems, but implemented with all various latent com methods from these papers and seeing how good they become?
    - Take sota MAS and evaluate them on qwen3/lama swarms
- How do we test this for closed-source models? Can we have a model that tries to approximate soft-tokens of close-source models at prediction time?
- Improve coding agents with latent communication
    - Problem: sub-agents not meant to communicate everything? Or just select the exact output?
        - Also, usually other type of models for sub-agents
- Improve recursive LMs with latent communication
- Improve memory systems with latent communication
    - Inter-agent communication can be replaced with methods above
    - Note taking systems that some memory systems integrate can also be improved with KV-cache -> already kinda done, but only in single agent contexts
    - Open weights SOTA memory systems to try improve: :
- Study interpretability in latent communication

Latent communication

LatentMAS (https://arxiv.org/pdf/2511.20639)

- They only test QA systems (even for code, the system has to one-shot it, as it has not access to external tools)
- Could be interesting to extend to more complex MAS systems like Claude Code, memory systems or even Recursive Language Models
    - Recursive language models: https://arxiv.org/pdf/2512.24601
- Is latent thought really worth it? Maybe more ablation on this system is required
- RecursiveMAS already exists: https://arxiv.org/html/2604.25917v1
    - But this refers to recursive agent systems, not the true RLM concept
- Their input-output distribution alignment is worth giving a look
    - What if we just replace it with soft tokens?
- Theorem 3.1 is worth taking a look at -> expressiveness of latent thought
    - Can there be anything done to simulate latent thought on closed-source models? Approximating soft tokens, etc.
- Why would you concatenate the whole KV cache to A2?
    - Could maybe just concat the very last state? Communication will become lossy and less importance might be given to just one “token.” Maybe I can modify the system / fine-tune to give more importance to this token?
        - Also consider that we keep Ks and Vs from every layer anyways
    - Theorem 3.3 is really interesting
    - What KV to share is an active area of research
        - Especially for single models -> perform compression on A1 before communication with A2
        - KVComm: uses a Gaussian prior to select only the relevant KV pairs https://arxiv.org/pdf/2510.03346 (selecting over layers vs. selecting over tokens -> what if both?)
            - Note that layers are residuals -> if we select A1 KV at one layer, information flows to the upstream layers

Soft-thinking (https://arxiv.org/pdf/2505.15778)

- Concept of weight-tying -> usually not on large models !!! (because restricts freedom of learning)
- Cold-stop looks like a big problem (although it physically makes sense)
    - What if we just pick the nearest “in distribution” token? (will we get the max token from softmax anyways?)
    - What if we pick 1000 important concept tokens (approximate) and add them to our vocab? -> and then for decoding -> apply same mechanism to bullet point above
        - Training on this is actually a very complex problem
- Recursive Approximation step losses a lot of accuracy because error grows exponentially -> an analysis of this is worth it on itself

Communicating Activations Between Language Model Agents (https://arxiv.org/pdf/2501.14082)

- Could make improvements for learning W? (the translation matrix between model A and B spaces)
- Information on last layer could throw away information that's not relevant to next token prediction -> that's why it makes sense to use it for inter-agent communication
- Model A does not generate any new tokens -> we just let it process the information
    - So is this harder to extend to usual MAS?
    - A is used purely to encode tokens -> could maybe use for memory systems for this reason
- K = J seems optimal

CIPHER (https://arxiv.org/pdf/2310.06272)

- Same concept as Soft Thinking paper, but used strictly for inter-agent communication, not reasoning —> what if we combine the two as an alternative to LatentMAS

Other Observations:

- A lot of attempts on KV-communication -> a grand comparison of all methods is needed
- Could study interpretability for such systems

Used systems that we could attempt improve with latent communication

- Memory systems
    - Can use KV cache to create a note-system. KV-cache as a plug-and-play for other agents who want to get context (already kinda done)
    - Mem0: https://github.com/mem0ai/mem0 (not multi-agent)
    - MIRIX (arXiv 2507.07957) -> is multi agent
    - MemArt, FlashMem, IndexMem, DeltaKV -> single-agent, but focus on KV cache note-taking systems
- Coding agents
    - Does not always make sense -> sub-agents sometimes have to be isolated -> have to figure out when they can be applied
    - Often, precise symbols have to be communicated, which latent communication blurs -> would again be interesting to develop a gate to differentiate between when latent communication is needed vs. when concrete information can be communicated
        - Actually, maybe not even a problem -> besides appending A1 KV-cache, always include a small sections (that A1 is instructed to generate at the end) of spelled out tokens
- RLMs (https://arxiv.org/pdf/2512.24601) -> not done; hard to do because sometimes direct quotes are needed -> what if we develop a gate to decide when we use KV communication vs. direct word communication?
    - Also, could this solve the depth problem in the RLM paper? -> No; paper proves that depth 1 is enough if root model trained well enough to handle the REPL environment

Memory

Papers from Yusen:

https://arxiv.org/pdf/2605.21951v1
https://arxiv.org/pdf/2506.07398

Shared-memory literature review for MAS

Actively maintained repos of MAS-memory systems

- https://github.com/ShanglinWu/Awesome-MAS-Memory
- https://github.com/Shichun-Liu/Agent-Memory-Paper-List

Benchmarks

- No agreed upon benchmark

G-Memory (https://arxiv.org/pdf/2506.07398)

- Self-evolving MAS is hard because single-agent memory systems are not generalizable to MAS
- 3 graphs: insights -> query -> interaction
- Organizational memory theory
- Interaction graph is very easy to build -> just all agent interactions within one query
- Edges connect related queries (either semantically, or when one query helped another)
- An insight node covers the queries that use it
- For a newly inserted query, expand via top-k similarity queries + 2-hop
    - Then, extract all insights that have significant overlapping sets with the set of queries
    - Get the interactions of the most relevant queries (and summarise them into essential steps for the given task)
        - QUESTION: why is it needed to keep this whole graph? Structure could be simplified
    - Now give each agent in MAS a relevant section from this memory (decided by another LLM)
- There are a lot of LLMs deciding stuff + feeding summaries into the MAS => try to insert latent alternatives???

MIRIX (https://arxiv.org/abs/2507.07957)

- Not for MAS, but uses a MAS for memory => even a better application for latent communication -> Could apply LatentMAS
- Stores memory in 6 different buckets (core, episodic, semantic, procedural, resource, knowledge vault)

RCR-Router: Efficient Role-Aware Context Routing for Multi-Agent LLM Systems with Structured Memory (https://arxiv.org/abs/2508.04903)

Collaborative Memory (https://arxiv.org/abs/2505.18279)

LegoMEM (https://arxiv.org/pdf/2510.04851)

- Decomposes past-query MAS traces into granular pieces that you plug into context of new queries
- But focuses on start topology MAS

Memory-R1 (https://arxiv.org/pdf/2508.19828)

- Not MAS; but same RL idea for LatentMem

Latent memory (for single agents and for MAS)

Mem0 (https://arxiv.org/pdf/2504.19413)

- not latent, but is memory SOTA (for general use case)
- At each step, collect the last pair of messages + a sliding window of m messages + a summary of the whole conversation (constantly updated by a special module) => generate n salient facts.
    - For each salient fact, search the db for the most s relevant memories (RAG) => 4 operations: UPDATE, NOOP, DELETE, ADD
- Also has a graph version for storing the DB
- Idea: try to implement latent communication + storing of ideas for their exact pipeline
- LOCOMO (https://snap-research.github.io/locomo/): just test the memory system on a set of predefined conversations

Titans (https://arxiv.org/pdf/2501.00663)

- full parametric, not interesting
- Read nested learning also from them
- Really popular paper

MemGen (https://arxiv.org/pdf/2509.24704)

- Memory trigger and weaver are just LoRa adapters for the base model powering the agent
- Memory trigger has different head from vocab projection => yields probability
    - Only at periods and commas (sentence granularity, Anthropic paper)
- Reward-adaptive penalty for the RL training of the trigger
- Weaver head gives a latent token sequence instead of a scalar
- Weaver only contains information from its training over agent traces
    - But can it be combined with RAG memory?
- This is STILL parametric memory!!! Only real difference is that it lives in a module
    - A way to fine-tune, but only invokes knowledge from fine-tuning when needed => does not cause model forgetting
    - Can this be extended for continual learning ??? => create a weaver / trigger for the weaver / trigger, when trying to add a new domain of knowledge

FlashMem (https://arxiv.org/abs/2601.05505)

- Builds on the fact that last layer retains rich trajectory information
- Measures attention entropy on last layer (last layer is more free of attention sinkholes + they mask known sinks anyways) to note model confusion and trigger model consolidation
    - Entropy threshold depends on model's entropy distribution across a val set
    - I wonder if this entropy concept can be applied to other memory systems?
- The memory consolidation used just h_t as input (summary of whole context)
    - Small decoder model (1 layer) that only trains for Query matrix (shares KV with backbone model)
    - M_1, m_2, m_3, …, m_k (no inter-latent attention)
    - These are introduced right away in the KV-cache => note how the next memory generation will inevitably attend to these "tokens"!
- Weight inheritance for training module
- FleshGen a bit worse than MemGen => but fleshmem is much faster
- How can this be extended to MAS?
    - IDEA: Measure entropy at system level maybe? -> or study what edges in the DAG led to higher entropy later?

LatentMem (https://arxiv.org/abs/2602.03036**)**

- Latent memory for MAS!!!
- Role-aware latent memories for agents
- Experience bank (raw interaction history from previous queries) and memory composer (trained through RL)
- Trajectory retrieval from the bank is just RAG here
- Composer produces K latent embeddings for each agent that will be appended to their context
    - Trained with LMPO (GRPO variant)
- Principle: for a scalable system to work, want not to handcraft insights (like G-Memory), but have a learned module
- IDEA: instead of summarizing traces from the bank, make a subselection of KV-caches from the bank!!!

Memory - continue

MAST (https://arxiv.org/pdf/2503.13657)

- Really important paper on why MAS fail => start from here if want to do research with taste
- Step repetition is 15% of failures
- [Theory of Mind]]Agents fail to communicate other agent's needs -> can this be solved with latent?
- **Multi-agent Graph-Attention** (https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p964.pdf) -> very old paper that could be revived
- **Uncertainty, Action** (http://erichorvitz.com/ftp/mixedin.pdf) -> only acting when confidence is high -> otherwise, gather information from others

How to measure when two agents are in disagreement?

MemEvolve (https://arxiv.org/pdf/2512.18746)

- similar in principle to AIM?
-

Sometimes, an agent knows something obvious about its task. It goes on and at some point measures something. It is not obvious to the agent that the measurement contradicts the obvious fact it previously acknowledged. How do layers look when this is happening? What is happening? Is this the true problem of agentic memory?

Separate questions

Separate questions:

- Can you predict how the KV cache would look for a model generating a certain text?
- Can you get next-token predictions for the newer model APIs? Can you just ask it to predict the next token? And then look at old models and see how that looks -> could be a quick experiment to do
- Could you gradient descent on the structure of a coding MAS to find the best coding MAS? Can use the skills.md autograd or something.

READ: Black box uncertainty estimation!!!

Just an interesting paper: https://arxiv.org/pdf/2604.28119

- Prerequisite about SAEs: https://arxiv.org/abs/2309.08600

Check out: **Guibin Zhang -> very good ideas for multi-agent studfd**

Shuicheng Yan -> competitor to Columbia

LatentMem paper from the same group

Experience vs memory

- Experience is about failing
- Memory is just about not forgetting

Single-agent latent memory -> few papers

Single-agent latent experience -> many papers

Start with problem in MAS

- Example: agents not knowing what others are doing

TODO: keep reading and classify -> summarize existing work -> make a visual

Visual = table with columns (problem: memory, experience) and lines (methods)

Ask question: why do MAS need memory? And why does it have to be latent?

Run experiments from g-memory paper and find failure cases.

Next time, make a slide where I give example of how memory can help

Look at theory of Mind for MAS paper!!!

Maybe instead of storing experience, we could store the status of the MAS

G-Memory + latent not a good idea because reproducing papers is a problem
    If i want to reproduce, do so from larger groups

Latent communication and latent memory both rely on the same problem -> why do MAS need them?

This paper if good for baselines: https://arxiv.org/pdf/2602.03036

For NLP paper, we care more about ideas, then www conf (etc) where they focus on details

A better idea than adding over papers is subtracting (plus, not minus)
- For g-mem for example

Nobody proposed blackboard for MAS

Any paper with MAS and auxiliary module can be helpful!!!
Try to do more lit review -> do catalogation and CHECK WHAT TYPE OF “MEMORY” THEY ARE TALKING ABOUT

MAS catalogue

Multi-Agent Systems (architectures)

ChatDev

- **1.0:** linear, specialized SWE agents and stages, passes on summary to the next interaction
- **MacNet:** random DAGs
    - Diminishing returns, but grokking at 1000+ agents
- **2.0**: just an orchestration platform

AutoGen

- Just a framework
- **AG2**: 2 agent RAG collaboration (or tool execution) loop
- **AG3**: solver / ground truth / executor agents
- **Megentic-One**: star-shaped, various tools

MetaGPT

- Chain execution, but memory (information) is added to a black board
- Uses SOPs for standardized communication and execution

DyLAN

- Organizes agents like an MLP
- Per task, importance scores are computed for edges -> unimportant nodes are pruned
- Time-series based MLP -> stops early at consensus

CAMEL

- A dyad just like A2
- Introduced inception prompting
- Not really relevant anymore
- But they do have a suite of more modern tools

Dynamic MAS

- G-Designer, GPTSwarm, AFlow, EvoFlow, MaAS
- Dynamic architecture directly attack the problem of intra-task memory (but not extra-task)
    - Although for some of these architectures, they are learned across multiple tasks => some extra-task knowledge is accumulated in the topology

Memory Systems natively adopted by MAS

MetaGPT-M

- Terminology developed by G-Memory
- Shared message board (recall how MetaGPT worked, with predefined roles)

ChatDev-M

- Terminology developed by G-Memory
- Intra-trial: passing context from previous round
- Extra-trial: simple query - solution storage

MacNet-M

- Terminology developed by G-Memory
- A lower bound, discarding inter-agent insights -> only saved query - answer pairs

Memory Bank

- Fancy blackboard which mimics anthropomorphic memory
- Old memories decay, unless reinforced by new tasks

Voyager

- Terminology developed by G-Memory (from the Voyager Minecraft agent)
- Also similar to Hermes Agent, basically developing skills as it interacts with the environment and discovers new things

Literature review:

- https://arxiv.org/pdf/2603.22386
- https://arxiv.org/pdf/2601.14192 (more comprehensive, includes memory for MAS)

Bank of Ideas

Bank of Ideas

- Maybe the solution is dynamic topology + memory?
- Do I care about intra-task or extra-task memory? Which is more relevant right now?
- Do I care about experience memory or context memory? Or are they the same?
- Memory research: single-agent (working vs. procedural) -> multi-agent -?-> evolution-multi-agent? (i.e., how do you improve the memory of evolution systems)
    - https://human-agent-society.github.io/CORAL/ (CORAL)
    - https://hive.rllm-project.com/ (HIVE)
    - This is regarding Yusen telling me to find a new field / research gap
- Paper about MAS which keeps persistent view of codebase: https://arxiv.org/pdf/2605.20563
- Giga dynamic MAS : https://arxiv.org/pdf/2605.09539 -> adapting both memory and topology (so already attacks the CORAL direction?)

TODO:

- Read more about various MAS architectures and issues
- What about multi-agentic evolution systems?
- Try Autogen (A2 or A3) and maybe ChatDev and maybe MacNet
- Study the benchmarks of LatentMAS and G-Memory
- Detect better gap in research -> better consolidate the direction that I want to pursue
    - Or maybe start with the structured literature review that Yusen was talking about
- SHOULD READ LEGOMEM (from Microsoft)
- Add JoyAgent and OAgent to reading list